

Рубежный контроль №2 (Вариант №20)

Задание №1

Правило $bc \rightarrow cb$

Это правило меняет местами соседние b и c , сдвигая c на одну позицию влево, и при этом это правило не меняет количества b и c . Из слов вида $b^p c^q$, где все b слева, а все c справа), последовательными применениями этого правила можно получить любую перестановку букв b и c

Правило $ac \rightarrow c^3$

Это правило уменьшает количество букв a на 1 и увеличивает число c на 2. В исходных базисных словах $a^n b^{n+k} c^k$ подстроки ac нет. Чтобы применить $ac \rightarrow c^3$, надо сначала при помощи $bc \rightarrow cb$ протащить хотя бы один c влево через все b , чтобы получить подстроку вида $a^n c \dots$

По условию, базисным словом является $a^n b^{n+k} c^k$. Рассмотрим различные случаи при разных вариантах применении имеющихся правил.

Случай А: правило $ac \rightarrow c^3$ не применялось

Меняется только порядок букв b и c , при этом сохраняя их изначальное количество:

$$|w|_b = n + k, \quad |w|_c = k$$

Тогда могут получиться все слова вида

$$L_1 = \{ a^n w \mid w \in \{b, c\}^*, |w|_b = n + k, |w|_c = k, \}.$$

Случай В: правило $ac \rightarrow c^3$ применили $t \geq 1$ раз

Это возможно только если $k \geq 1$, т.к. чтобы создать подстроку ac , нужен хотя бы один c .

Для применения этого правила можно протащить один c сразу после a^n . В результате получим $a^n c b^{n+k} c^{k-1}$.

Теперь можно применить $ac \rightarrow c^3$ t раз. Каждое применение уменьшает число a на 1 и добавляет 2 к числу c . После t применений:

$$|w|_a = n - t \quad \text{и появляется блок } c^{2t+1}.$$

Оставшаяся часть слова состоит из букв b и c ($|w|_b = n + k$, $|w|_c = k - 1$) в любом порядке (их можно переставлять с помощью правила $bc \rightarrow cb$).

Итак,

$$L_2 = \left\{ a^{n-t} c^{2t+1} w \mid n \geq 0, k \geq 1, 1 \leq t \leq n, \right. \\ \left. w \in \{b, c\}^*, |w|_b = n + k, |w|_c = k - 1 \right\}.$$

Отсюда следует, что

$$L = L_1 \cup L_2.$$

Представление языка в виде PDA

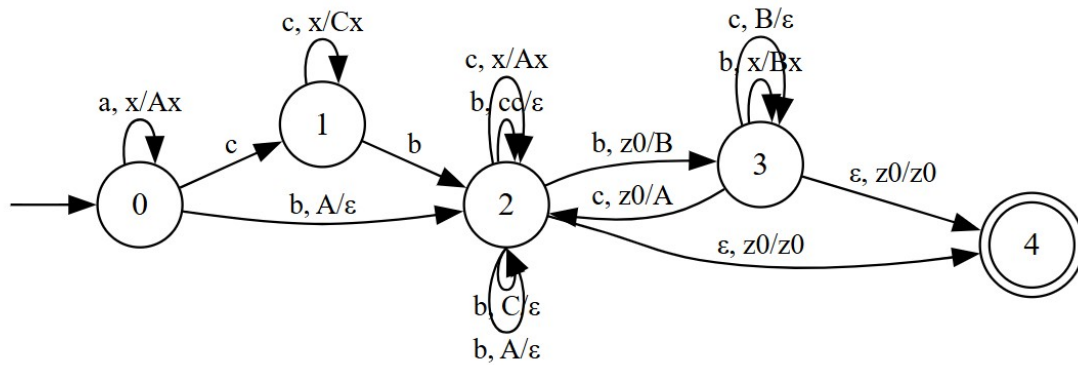


Рис. 1: PDA-автомат

Проверка регулярности языка

Рассмотрим регулярный язык

$$R = a^*b^*,$$

Пересеку его с имеющимся языком L

$$L \cap R = \{a^n b^n \mid n \geq 0\}.$$

Язык

$$\{a^n b^n \mid n \geq 0\}$$

не является регулярным. Это можно легко доказать в том числе используя таблицу классов эквивалентности. Если бы L был регулярным, то и пересечение $L \cap R$ было бы регулярным. Следовательно, язык L не является регулярным

Задание №2

Рассмотрим случаи, в которых точно можно быть уверенным, что слово будет принадлежать языку

1. Пусть $|w_1| = 2$ и w_1^R найдено больше, чем 1 раз. Тогда из этих $(w_1)^R$ можно выбрать подходящий вариант, чтобы было выполнено условие $|z_1| \neq |z_2|$.
2. Пусть $|w_1| = 3$ и найдено w_1^R . Тогда при необходимости вместо $|w_1| = 3$ можно выбрать $|w_1| = 2$, в результате чего точно будет выполняться условие $|z_1| \neq |z_2|$.

Случай, где пункты 1 и 2 не выполняются

Если пункты 1 и 2 не выполняются, то тогда известно, что $|w_1| = 2$ и w_1^R единственно. Для решения этого случая можно нарисовать DPDA.

Рассмотрим все возможные тройки, с которых начинается слово (aaa, aab, aba, abb, baa, bab, bba, bbb):

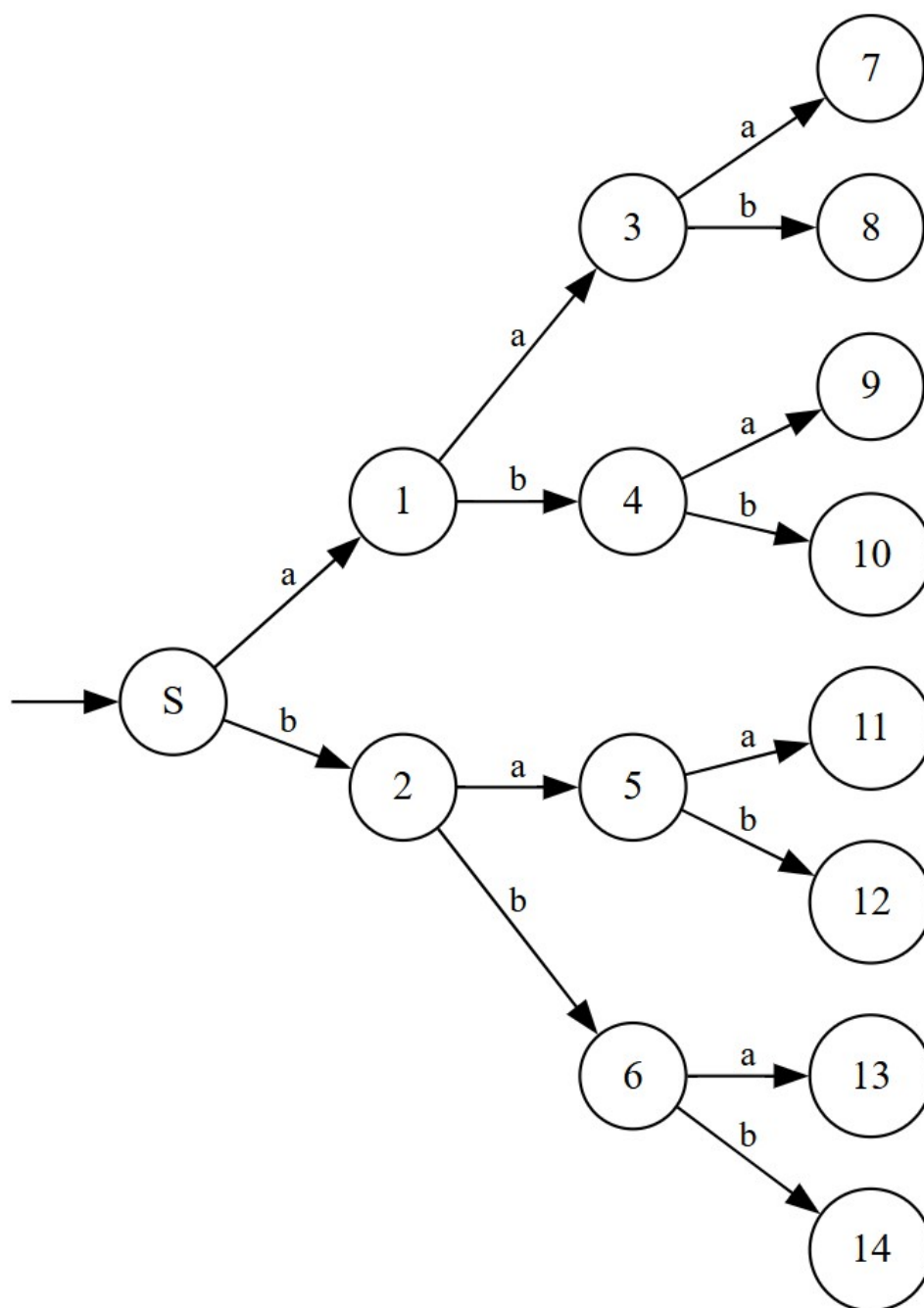


Рис. 2: Начало общего DPDA

Каждая из этих вершин будет иметь уникальный DPDA для поиска развёрнутого пути до этой вершины. Так, одна из веток (для тройки aba) будет выглядеть следующим образом:

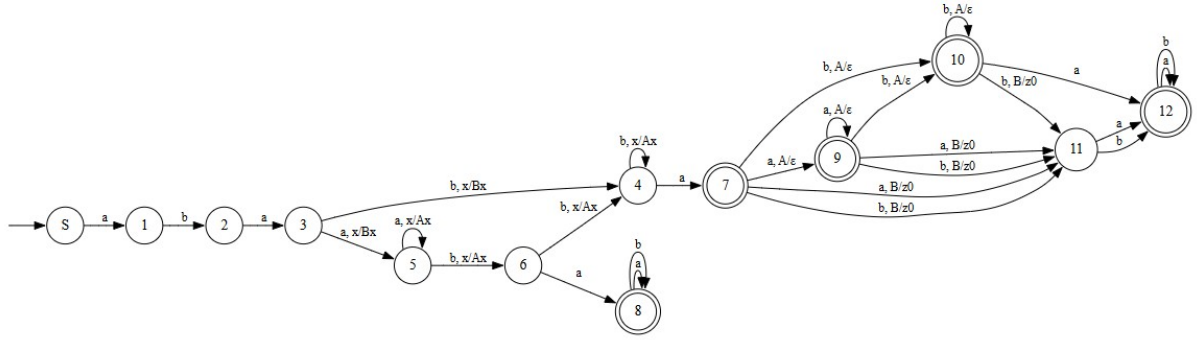


Рис. 3: ветвь DPDA по пути aba

Используя те же идеи, можно построить DPDA для каждой тройки. В результате этого получится общий DPDA для этого языка.

Таким образом можно утверждать, что язык является DCFL.

Проверка LL-свойства

Рассмотрим слово, входящее в исходный язык: $abab^{(n+k)}aa$. Для n подбирается такое значение, что после чтения n символов стек будет содержать не менее, чем $k + 3$ стековых символа. Рассмотрим еще одно слово, входящее в исходный язык: $abab^{(n+k)}aab^{(n+k+1)}$. Для данного слова первое слово $abab^{(n+k)}aa$ является префиксом, к которому был добавлен суффикс $b^{(n+k+1)}$. При чтении фрагмента $b^k aa$ автомат может снять со стека не более $k + 2$ символов. Тогда при чтении слова $abab^{(n+k)}aa$ стек гарантированно будет непустым, и автомат не сможет корректно завершить работу. Получается неожиданный конец при том, что префикс $abab^{(n+k)}aa$ продолжим до слова из L . Следовательно, язык не может быть $LL(k)$.

Задание №3

Из первых двух правил для S :

$$S \rightarrow TbS, \quad S \rightarrow T$$

следует, что любой вывод из S имеет вид

$$\begin{aligned} S &\Rightarrow TbS \\ &\Rightarrow TbTbS \\ &\Rightarrow \dots \\ &\Rightarrow (Tb)^n T. \end{aligned}$$

То есть любое слово языка представимо в форме

$$w = u_0 b u_1 b u_2 b \dots b u_n, \quad \text{где } u_i \in L(T).$$

Важно отметить, что буквы b между блоками это именно те b , которые получаются из правила $S \rightarrow TbS$.

Используем семантику правила

$$S \rightarrow T b S \quad ; \quad S_0.a := S_1.a + 1, \quad S_1.a == T.a.$$

то есть:

- атрибут хвоста $S_1.a$ должен совпасть с атрибутом текущего $T.a$;
- атрибут “всего” на 1 больше атрибута хвоста.

Если записать слово как $u_k b u_{k-1} b \dots b u_0$, то получается:

- правый блок u_0 (последний T) получает некоторое значение

$$T_0.a = f;$$

- блок перед ним обязан иметь то же самое значение (потому что хвост равен текущему $T.a$):

$$T_1.a = f;$$

- следующий левее уже должен быть на 1 больше:

$$T_2.a = f + 1;$$

- дальше по цепочке каждый левый блок на 1 больше предыдущего:

$$T_3.a = f + 2, \dots, T_k.a = f + (k - 1).$$

Правило $T \rightarrow aTa$

Единственное правило для T , которое порождает a , имеет вид

$$T \rightarrow aTa.$$

Следовательно, каждое применение правила $T \rightarrow aTa$ добавляет ровно две буквы a . Поэтому любое слово, выводимое из T и состоящее только из букв a , имеет вид

$$a^{2m}$$

для некоторого $m \geq 0$. То есть степень всегда кратна 2.

Чтобы вывести слово a^{2m} из T , правило $T \rightarrow aTa$ необходимо применить ровно m раз. После чего вывод заканчивается правилом $T \rightarrow \varepsilon$.

Для правила $T \rightarrow aTa$ возможны две семантики:

$$T_0.a := T_1.a + 1 \quad \text{или} \quad T_0.a := T_1.a + 2.$$

Значит, итоговое значение $T.a$ на слове a^{2m} равно сумме m слагаемых, каждое из которых равно 1 или 2. Отсюда:

$$\min T.a = m \quad (\text{если все } m \text{ раз брать } +1), \quad \max T.a = 2m \quad (\text{если все } m \text{ раз брать } +2),$$

таким образом

$$T.a \in [m, 2m].$$

Выбор слова для накачки

Рассмотрим слово

$$w_n = a^{2n+2} b a^{4n} b a^{2n}.$$

Оно имеет вид $T_2 b T_1 b T_0$, где

$$T_0 \Rightarrow a^{2n}, \quad T_1 \Rightarrow a^{4n}, \quad T_2 \Rightarrow a^{2n+2}.$$

Следовательно, число применений правила $T \rightarrow aTa$ равно:

$$\text{для } T_0 : n \text{ раз,} \quad \text{для } T_1 : 2n \text{ раз,} \quad \text{для } T_2 : n + 1 \text{ раз.}$$

Чтобы слово входило в язык, должны существовать такие значения, что

$$T_0.a = f, \quad T_1.a = f, \quad T_2.a = f + 1.$$

Так как $T_0 \Rightarrow a^{2n}$, то по описанному ранее имеем, что

$$T_0.a \in [n, 2n].$$

Так как $T_1 \Rightarrow a^{4n} = a^{2 \cdot (2n)}$, то по описанному ранее имеем, что

$$T_1.a \in [2n, 4n].$$

$T_0.a = T_1.a = f$, поэтому

$$f \in [n, 2n] \cap [2n, 4n] = \{2n\},$$

Таким образом происходит фиксация значения для $T_0.a$ и $T_1.a$

$T_2 \Rightarrow a^{2n+2} = a^{2 \cdot (n+1)}$, тогда

$$T_2.a \in [n+1, 2n+2].$$

Поскольку

$$2n+1 \in [n+1, 2n+2],$$

то нужное значение для $T_2.a$ действительно допустимо, и при этом сохраняется пересечение с пересечением $T_1.a$ и $T_0.a$

Необходимо добиться

$$T_0.a = 2n, \quad T_1.a = 2n, \quad T_2.a = 2n+1.$$

Для $T_0 \Rightarrow a^{2n}$. Правило $T \rightarrow aTa$ применяется n раз. Чтобы получить $T_0.a = 2n$, на каждом из n применений берём семантику $+2$:

Для $T_1 \Rightarrow a^{4n}$. Правило $T \rightarrow aTa$ применяется $2n$ раз. Чтобы получить $T_1.a = 2n$, на каждом из $2n$ применений берём семантику $+1$:

Для $T_2 \Rightarrow a^{2n+2}$. Правило $T \rightarrow aTa$ применяется $n+1$ раз. Чтобы получить $T_2.a = 2n+1$, берём n раз семантику $+2$ и один раз семантику $+1$:

Следовательно,

$$w_n = a^{2n+2} b a^{4n} b a^{2n} \in L.$$

Накачка

1. Отрицательная накачка внутри a^{2n+2}

Первый блок становится короче: $a^{2n+2} \mapsto a^{2n+2-k}$. Для такого блока возможные значения атрибута лежат в диапазоне $T_2.a \in [n+1-k, 2(n+1-k)]$, то есть максимум $\leq 2n$. Т.к. для этого слова обязательно должно быть $T_2.a = 2n+1$ - слово выходит из языка.

2. Положительная накачка внутри a^{4n}

Второй блок становится длиннее: $a^{4n} \mapsto a^{4n+k}$. Тогда минимально возможное значение становится $T_1.a \geq 2n+k > 2n$. А правый блок a^{2n} даёт значения только из $[n, 2n]$, поэтому равенство $T_1.a = T_0.a$ выполнить нельзя — слово выходит из языка.

3. Отрицательная накачка внутри a^{2n}

Третий блок становится короче: $a^{2n} \mapsto a^{2n-k}$. Тогда максимально возможное значение $T_0.a \leq 2n-k < 2n$, а средний блок a^{4n} даёт значения $T_1.a \in [2n, 4n]$, то есть $T_1.a \geq 2n$. Следовательно, равенство $T_1.a = T_0.a$ невозможно — слово выходит из языка.

4. Накачка попарно внутри a^{2n+2} и a^{4n}

Положительная накачка блока a^{4n} увеличивает минимальное и максимальное значения, и тогда снова $T_1.a$ становится больше $2n$, а $T_0.a$ остаётся $\leq 2n$ - слово выходит из языка.

5. Накачка попарно внутри a^{4n} и a^{2n}

Положительная накачка блока a^{2n} при больших значениях приведет к тому, что минимальное значение $T_0.a$ будет больше минимального значения для $T_1.a$, в результате чего между ними пропадёт пересечение - слово выходит из языка

Противоречия по b (из правила $T \rightarrow bT$)

Нужно проверить, может ли какая-то из букв b в этом слове появиться за счёт правила $T \rightarrow bT$. Если буква b появилась именно из правила $T \rightarrow bT$ и перед этой b в итоговом слове стоят только a , то эти a могли появиться только из обёрток $T \rightarrow aTa$, а значит ровно столько же a обязано стоять и в конце того же T -блока.

Первая b (между a^{2n+2} и a^{4n}) получена правилом

Чтобы попытаться получить слово w при указанных условиях единственный порядок действий выглядит следующим образом:

$$\begin{aligned} S &\rightarrow TbS \rightarrow TbT \rightarrow Tba^nTa^n \rightarrow Tba^{2n} \rightarrow a^{2n+2}Ta^{2n+2}ba^{2n} \rightarrow \\ &\rightarrow a^{2n+2}bTa^{2n+2}ba^{2n} \rightarrow a^{2n+2}ba^{n-1}Ta^{n-1}a^{2n+2}ba^{2n} \rightarrow a^{2n+2}ba^{4n}ba^{2n} \end{aligned}$$

при выполнении такого порядка действий промежутков значений $T_0.a = [n, 2n]$, тогда как промежутков значений $T_1.a = [2n+2+2 \cdot (n-1), 4n+4+2 \cdot (2n-2)] = [4n, 8n]$. Значения блоков должны быть равны, а поскольку промежутки не имеют пересечения, это слово в языке таким образом не могло быть получено

Вторая b (между a^{4n} и a^{2n}) получена правилом

Чтобы попытаться получить слово w при указанных условиях единственный порядок действий выглядит следующим образом:

$$\begin{aligned} S &\rightarrow TbS \rightarrow TbT \rightarrow a^{n+1}Ta^{n+1}bT \rightarrow a^{2n+2}ba^{4n}Ta^{4n} \\ &\rightarrow a^{2n+2}ba^{4n}bTa^{4n} \rightarrow a^{2n+2}ba^{4n}ba^{4n} \end{aligned}$$

$4n > 2n$, следовательно это слово в языке таким образом не могло быть получено

Обе b получены правилом

Чтобы попытаться получить слово w при указанных условиях единственный порядок действий выглядит следующим образом:

$$\begin{aligned} S &\rightarrow T \rightarrow a^{2n+2}Ta^{2n+2} \rightarrow a^{2n+2}bTa^{2n+2} \rightarrow a^{2n+2}ba^{4n}Ta^{4n}a^{2n+2} \\ &\rightarrow a^{2n+2}ba^{4n}bTa^{6n+2} \rightarrow a^{2n+2}ba^{4n}ba^{6n+2} \end{aligned}$$

$6n+2 > 2n$, следовательно это слово в языке таким образом не могло быть получено

Аналогичные результаты получаются при исследовании противоречий для каждой накачки.

Таким образом можно сделать вывод, что данный язык не является КС-языком